

LeetCode 444 — Sequence Reconstruction (Premium)

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 30, 2026

Problem Statement

You are given:

- An original sequence `org = [1, 2, 3, 4]`
- A list of subsequences:

```
seqs = [ [1,2], [2,3], [3,4] ]
```

Each subsequence tells you:

"These numbers must appear in this order."

Can these subsequences reconstruct exactly one sequence, and is that sequence equal to `org` ?

Example 1: Multiple Valid Answers

```
org = [1,2,3]
```

```
seqs =
```

```
[1,2]
```

```
[1,3]
```

Graph:

```
graph TD; 2 --- slash["/"]; slash --- 1; 1 --- backslash["\\"]; backslash --- 3
```

Topological orders are

1 2 3 AND 1 3 2

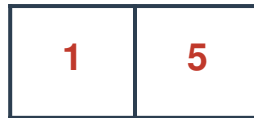
Multiple valid answers.

Return **false**

Kahn's Algorithm Logic

suppose queue is

Queue



Uh-oh.

You can pick 1 or 5 . Both are valid.

Immediately there are **2 different topological orders**.

Therefore ordering is not unique.

During Kahn, if at any moment `q.size() > 1` then there is more than one valid topological ordering.

Return **false**

Handling Duplicate Edges

Suppose `org = [1,2,3]`

Now suppose `seqs =`

`[1,2]`

`[1,2,3]`

`[2,3]`

`[1,2]` appeared twice. If we blindly build graph:

First sequence

`adj[1].push_back(2); indeg[2]++;`

Now `indeg[2]=1` . Good.

Second sequence Again

`adj[1].push_back(2); indeg[2]++;`

Now `indeg[2]=2`

But actually There is still only $1 \rightarrow 2$ one dependency.

Edge Case: Empty Sequences

Imagine `org = [1]`

`seqs = []`

No subsequences are given.

Should we return `true` or `false`

Answer is **false**

Why? Because we have zero information. Nobody told us anything about the sequence.

Solution (C++)

```
class Solution {
public:
    bool sequenceReconstruction(vector<int>& org,
vector<vector<int>>& seqs) {

        int n=org.size();

        //same template as course schedule
        //bas ek extra dikkat he
        //same edge baar baar aa sakta he
        //usse indegree galat ho jayega
        //isliye duplicate edge ignore kreng

        vector<vector<int>> adj(n+1);
        vector<int> indeg(n+1,0);

        unordered_set<string> edge;

        bool found=false;

        for(auto &seq:seqs){

            for(auto &x:seq){

                if(x<1 || x>n) return false;

                found=true;
            }

            for(int i=1;i<seq.size();i++){

                int u=seq[i-1];
                int v=seq[i];

                string currEdge=to_string(u)+"#"+to_string(v);

                //already graph me he
                if(edge.find(currEdge)!=edge.end()){

                    continue;
                }

                edge.insert(currEdge);

                adj[u].push_back(v);
                indeg[v]++;
            }
        }

        return found;
    }
};
```

```

    }
}

if(!found) return false;

queue<int> q;

for(int i=1;i<=n;i++){

    if(indeg[i]==0){

        q.push(i);

    }
}

vector<int> ans;

while(!q.empty()){

    //agar ek se zyada choices he
    //to unique ordering possible nahi

    if(q.size()>1){

        return false;

    }

    int curr=q.front();
    q.pop();

    ans.push_back(curr);

    for(auto &it:adj[curr]){

        indeg[it]--;

        if(indeg[it]==0){

            q.push(it);

        }

    }

}

if(ans.size()!=n) return false;

return ans==org;

}

};

```